



Sistemas de Tiempo Real (STR)



Un sistema de tiempo real es un sistema al que se le exige responder a peticiones específicas dentro de **límites de tiempo estrictamente predefinidos**, ya procedan de un usuario o de un sensor externo.

Explicación

Un STR no solo debe producir **resultados lógicamente correctos**, sino que debe hacerlo **dentro de una ventana temporal garantizada**.

Esto es lo que diferencia a un sistema concurrente general de un sistema de tiempo real:

En un STR, la condición de progreso **no es "eventual"**, sino *"dentro de una cota temporal obligatoria"*. Esto es una excepción a la asunción del progreso finito.

Estas cotas temporales reciben el nombre de **ligaduras temporales**. Si una acción se ejecuta fuera de su deadline, aunque el resultado sea correcto, **el sistema se considera erróneo**.

Ejemplos típicos:

- Sistemas de control de tráfico aéreo.
- Sistemas de monitorización hospitalaria.
- Controladores industriales.

Un STR puede ser lógicamente correcto pero **temporalmente incorrecto**, lo que lo vuelve inválido.

Propiedades

Un sistema de tiempo real debe garantizar:

Determinismo temporal

Responder siempre dentro de los límites de tiempo establecidos, sin depender de:

- la carga del sistema,
- el algoritmo de planificación,
- el número de procesadores,
- interacciones entre tareas.

Predictabilidad

El comportamiento, el rendimiento y el tiempo de respuesta deben ser **predecibles**.

Fiabilidad

Son sistemas concurrentes, por lo que deben cumplir también:

- exclusión mutua,
- ausencia de interbloqueo,
- ausencia de inanición.

Flujo típico de un STR

1. Lectura de sensores.
2. Computación.
3. Activación de actuadores.
4. Actualización del sistema controlado.

Este ciclo se repite de forma periódica, aperiódica o esporádica dependiendo del tipo de tarea.

Tipos de STR

Tiempo Real Estricto (Hard Real-Time)

El deadline **debe cumplirse siempre**.

Control de inyección de combustible en un motor.
La corrección temporal es vital: 2 ms tarde podría dañar el motor.

Tiempo Real Flexible / No Estricto (Soft Real-Time)

Se intenta cumplir los deadlines, pero **no es catastrófico** incumplir algunos.

Tiempo Real Firme (Firm Real-Time)

Los resultados tardíos pierden valor, pero el sistema no colapsa.

Streaming de vídeo.
Un frame llega tarde → pequeño salto visual → no es crítico.

Tareas en un STR

Las tareas representan el comportamiento del sistema. Pueden ser:

Periódicas

Se activan cada T unidades de tiempo
(Ej: leer sensor cada 10 ms).

Aperiódicas

Dependen de un evento **no regular**.
(Ej: clic del usuario).

Esporádicas

Eventos no regulares pero con **mínimo tiempo garantizado entre activaciones**.
(Ej: alarma cuando la temperatura supera 80°).

Recursos

Elementos del sistema requeridos para ejecutar tareas:

- CPU(s)
- Memoria
- Sensores / actuadores
- Variables compartidas

Planificación en STR

La planificación determina **en qué orden** y **cuándo** se ejecutan las tareas, garantizando que **todas cumplan sus deadlines**.

1. Planificación Estática Sin Prioridades (Cíclica)

No hay concurrencia.

Las tareas se colocan secuencialmente en un **hiperperíodo**.

Se usa en sistemas muy simples y altamente deterministas.

2. Planificación con Prioridades

Prioridades Estáticas

Cada tarea recibe una prioridad fija basada en su período.

Rate Monotonic (RM)

- Cuanto **menor** es el período → **mayor prioridad**.
- Es óptimo para tareas periódicas independientes.

Ejemplo **RM**:

T1 → período = 10ms → mayor prioridad

T2 → período = 40ms

→ **T1** siempre se ejecuta antes que **T2**.

Prioridades Dinámicas

Las prioridades cambian en tiempo de ejecución.

Earliest Deadline First (EDF)

- La tarea con **deadline más cercano** obtiene mayor prioridad.
- Muy eficiente: si un conjunto de tareas puede planificarse, EDF lo consigue.

Ejemplo **EDF**:

En un instante:

D1 = 8 ms

D2 = 3 ms

→ **T2** tiene mayor **prioridad** (deadline más urgente).

Limitaciones de Java para los STR

Java estándar **no está diseñado para tiempo real**, por:

- GC impredecible,
- falta de control sobre memoria física,
- planificación no determinista,
- prioridad no garantizada,
- sincronización con latencias no acotadas,
- respuestas asíncronas no deterministas.

Para resolver esto se crea la **RTSJ (Real-Time Specification for Java)**.

RTSJ

Cambia la JVM para dar soporte a sistemas de tiempo real.

Introduce:

- Nuevos tipos de memoria:

`HeapMemory` , `ScopedMemory` , `ImmortalMemory` .

- Nuevos tipos de hilos: `RealtimeThread` .

- Planificación por **prioridades fijas con 28 niveles**.
- Control preciso sobre eventos asíncronos.

```
RealtimeThread rt = new RealtimeThread();  
rt.start();
```

Los parámetros de planificación se gestionan mediante:

- `SchedulingParameters`
- `PriorityParameters`